

Sintaxis de Laravel - Apuntes Completos

1. Consola de Comandos Artisan

1.1 ¿Qué es Artisan?

Artisan es la interfaz de línea de comandos incluida en Laravel que automatiza numerosas tareas habituales en el desarrollo de aplicaciones web. Su objetivo es hacer más eficiente el trabajo del desarrollador, evitando la creación manual de archivos y la escritura repetitiva de código básico.

1.2 Ventajas de usar Artisan

La principal ventaja de Artisan es la automatización. Por ejemplo, si quisiéramos crear un controlador manualmente, tendríamos que:

1. Navegar hasta el directorio `/app/Http/Controllers`
2. Crear un archivo con el nombre apropiado
3. Escribir o copiar el esqueleto básico del controlador
4. Asegurarnos de que la sintaxis sea correcta
5. Añadir los espacios de nombres necesarios

Con Artisan, todo esto se reduce a un único comando:

```
php artisan make:controller NombreController
```

1.3 Comandos Principales de Artisan

Comandos de información y ayuda

```
php artisan list
```

- Muestra un listado completo de todos los comandos disponibles en tu instalación de Laravel.
- Útil para explorar las capacidades de Artisan o recordar la sintaxis de comandos específicos.

```
php artisan help [comando]
```

- Muestra información detallada sobre un comando específico.
- Ejemplo: `php artisan help make:controller`

Comandos de base de datos

```
php artisan db:migrate
```

- Ejecuta las migraciones pendientes, creando o modificando la estructura de la base de datos.
- Las migraciones son archivos que definen cambios en el esquema de la base de datos de forma programática.
- Permite mantener un control de versiones sobre la estructura de la base de datos.

```
php artisan db:seed
```

- Ejecuta los seeders para poblar la base de datos con datos de prueba predefinidos.
- Muy útil durante el desarrollo para tener datos con los que trabajar.

```
php artisan migrate:rollback
```

- Revierte la última ejecución de migraciones.

- Permite deshacer cambios en la base de datos de forma controlada.

php artisan migrate:fresh

- Elimina todas las tablas y ejecuta todas las migraciones desde el principio.
- Útil para resetear completamente la base de datos durante el desarrollo.

php artisan migrate:refresh

- Revierte todas las migraciones y las ejecuta de nuevo.
- Similar a `migrate:fresh` pero usando rollback en lugar de eliminar tablas.

Comandos de creación (make)

php artisan make:migration NombreMigracion

- Crea un archivo de migración en el directorio `database/migrations` .
- Las migraciones definen la estructura de las tablas de la base de datos.
- Ejemplo: `php artisan make:migration create_users_table`

php artisan make:seeder NombreSeeder

- Crea un archivo seeder en `database/seeds` .
- Los seeders se utilizan para insertar datos de prueba en las tablas.
- Ejemplo: `php artisan make:seeder UserSeeder`

php artisan make:controller NombreController

- Crea un controlador básico en `app/Http/Controllers` .
- El controlador estará vacío pero con la estructura correcta.

php artisan make:model NombreModelo

- Crea un modelo Eloquent en `app/Models` .
- Los modelos representan las tablas de la base de datos y permiten interactuar con ellas.

php artisan make:middleware NombreMiddleware

- Crea un middleware para filtrar peticiones HTTP.
- Los middlewares se usan para autenticación, logging, CORS, etc.

php artisan make:request NombreRequest

- Crea una clase de validación de formularios.
- Permite centralizar la lógica de validación fuera de los controladores.

Comandos de información sobre rutas

php artisan route:list

- Muestra una tabla con todas las rutas definidas en la aplicación.
- Incluye información sobre el método HTTP, URI, nombre y controlador asociado.
- Extremadamente útil para depurar problemas de enrutamiento.

php artisan route:cache

- Crea un archivo de caché con todas las rutas para mejorar el rendimiento.
- Recomendado en producción, pero debe limpiarse durante el desarrollo si se modifican rutas.

php artisan route:clear

- Elimina el archivo de caché de rutas.

Comandos de configuración inicial

```
php artisan key:generate
```

- Genera una clave de encriptación aleatoria y segura.
- Se almacena en el archivo `.env` bajo la variable `APP_KEY`.
- Este comando DEBE ejecutarse al configurar una nueva aplicación Laravel.
- Laravel no funcionará correctamente sin esta clave, ya que se usa para cifrar sesiones, contraseñas y otros datos sensibles.

Comandos de optimización

```
php artisan config:cache
```

- Combina todos los archivos de configuración en uno solo para mejorar el rendimiento.

```
php artisan cache:clear
```

- Limpia la caché de la aplicación.

```
php artisan view:clear
```

- Limpia las vistas compiladas de Blade.

1.4 Creación de comandos personalizados

Laravel también permite crear tus propios comandos Artisan:

```
php artisan make:command NombreComando
```

Esto crea un archivo en `app/Console/Commands` donde puedes definir la lógica de tu comando personalizado.

2. El Enrutador (Router)

2.1 ¿Qué es el enrutador?

El enrutador de Laravel es el componente responsable de capturar las URL solicitadas por el usuario y traducirlas en invocaciones de métodos específicos en los controladores. Es el punto de entrada de todas las peticiones HTTP a la aplicación.

El archivo principal de enrutamiento es `/routes/web.php` para rutas web tradicionales, y `/routes/api.php` para APIs.

2.2 Funcionamiento básico

Cuando un usuario solicita una URL como `https://mi-servidor.com/usuario/eliminar/12`, el enrutador:

1. Analiza la URL y la divide en segmentos: `["usuario", "eliminar", "12"]`
2. Busca una ruta coincidente en el archivo de rutas
3. Extrae variables de la URL si están definidas (como el `12` en este ejemplo)
4. Invoca el método del controlador correspondiente
5. Pasa las variables extraídas como parámetros al método del controlador

2.3 Formas de definir rutas

Forma 1: Closure o función anónima

La forma más simple de responder a una ruta es usar una función anónima directamente:

```
Route::get('/hola', function() {  
    return "Hola, mundo";  
});
```

Esta forma es útil para rutas muy simples, pero no se recomienda para aplicaciones complejas porque mezcla lógica con enrutamiento.

Forma 2: Invocación de un método de controlador

```
Route::get('/hola', 'HolaController@show');
```

Esta es la forma tradicional (Laravel 7 y anteriores). Invoca el método `show()` del controlador `HolaController`.

Nota importante para Laravel 8 y superiores: Es necesario configurar el espacio de nombres en `/app/Providers/RouteServiceProvider.php`:

```
// En RouteServiceProvider.php  
$this->routes(function () {  
    Route::prefix('api')  
        ->middleware('api')  
        ->namespace('App\Http\Controllers')  
        ->group(base_path('routes/api.php'));  
  
    Route::middleware('web')  
        ->namespace('App\Http\Controllers')  
        ->group(base_path('routes/web.php'));  
});
```

Forma 3: Sintaxis moderna con clase de controlador

Desde Laravel 8, la sintaxis recomendada es:

```
Route::get('/hola', [HolaController::class, 'show']);
```

O para controladores invocables (con método `__invoke()`):

```
Route::get('/hola', HolaController::class);
```

2.4 Rutas con parámetros

Parámetros obligatorios

```
Route::get('/usuario/{id}', [UserController::class, 'show']);
```

El controlador recibirá el parámetro:

```
public function show($id) {  
    return "Mostrando usuario con ID: " . $id;  
}
```

Parámetros opcionales

```
Route::get('/usuario/{nombre?}', [UserController::class, 'show']);
```

El controlador debe manejar el caso en que el parámetro no exista:

```
public function show($nombre = 'Invitado') {  
    return "Hola, " . $nombre;  
}
```

Múltiples parámetros

```
Route::get('/usuario/{id}/post/{postId}', [UserController::class, 'showPost']);
```

```
public function showPost($id, $postId) {  
    return "Usuario $id - Post $postId";  
}
```

2.5 Rutas con nombre

Una práctica muy recomendable es asignar nombres a las rutas:

```
Route::get('/contactar', [ContactController::class, 'show'])->name('contact');
```

Ventajas de las rutas con nombre:

1. **Mantenibilidad:** Si cambias la URL, no necesitas modificar todas las referencias en el código.
2. **Legibilidad:** El nombre describe la función de la ruta.
3. **Generación de URLs:** Puedes generar URLs usando el nombre en lugar de la ruta literal.

Ejemplo de uso en vistas:

```
<a href="{{ route('contact') }}">Contactar</a>
```

Si cambias la ruta de `/contactar` a `/acerca-de`, solo modificas el enrutador y todos los enlaces se actualizarán automáticamente.

2.6 Verbos HTTP

El enrutador soporta todos los verbos HTTP estándar:

GET - Obtener recursos

```
Route::get('/usuarios', [UserController::class, 'index']);
```

Se usa para solicitar información al servidor sin modificarla. Es el verbo por defecto cuando accedes a una URL en el navegador.

POST - Crear recursos

```
Route::post('/usuarios', [UserController::class, 'store']);
```

Se usa para enviar datos al servidor, típicamente desde formularios, para crear nuevos recursos.

PUT - Actualizar recursos completos

```
Route::put('/usuarios/{id}', [UserController::class, 'update']);
```

Se usa para actualizar un recurso completo, enviando todos sus datos.

PATCH - Actualizar recursos parcialmente

```
Route::patch('/usuarios/{id}', [UserController::class, 'update']);
```

Similar a PUT, pero se usa cuando solo se actualizan algunos campos del recurso.

DELETE - Eliminar recursos

```
Route::delete('/usuarios/{id}', [UserController::class, 'destroy']);
```

Se usa para eliminar un recurso del servidor.

Múltiples verbos en una ruta

```
Route::match(['get', 'post'], '/form', [FormController::class, 'handle']);
```

```
Route::any('/cualquier-verbo', [Controller::class, 'method']);
```

2.7 Limitación de verbos PUT, PATCH y DELETE en HTML

HTML estándar solo soporta los métodos GET y POST en formularios. No puedes escribir:

```
<form method="PUT"> <!-- Esto NO funciona -->
```

Laravel soluciona esto mediante la directiva `@method` de Blade:

```
<form action="/usuarios/{{ $id }}" method="POST">
  @csrf
  @method('PUT')
  <!-- Campos del formulario -->
</form>
```

O usando el helper HTML:

```
<form action="/usuarios/{{ $id }}" method="POST">
  @csrf
  {{ method_field('DELETE') }}
</form>
```

2.8 Orden de las rutas

El orden en que se definen las rutas es crítico. Laravel evalúa las rutas en orden secuencial y usa la primera que coincida.

Ejemplo de error común:

```
// MAL - La segunda ruta nunca se ejecutará
Route::get('usuario/{nombre}', [UserController::class, 'show']);
Route::get('usuario/crear', [UserController::class, 'create']);
```

Cuando solicites `/usuario/crear`, Laravel coincidirá con la primera ruta y buscará un usuario llamado "crear".

Solución correcta:

```
// BIEN - Las rutas específicas van primero
Route::get('usuario/crear', [UserController::class, 'create']);
Route::get('usuario/{nombre}', [UserController::class, 'show']);
```

Regla general: Las rutas más específicas deben ir antes que las rutas más genéricas.

2.9 Agrupación de rutas

Laravel permite agrupar rutas que comparten características comunes:

Por prefijo

```
Route::prefix('admin')->group(function () {
    Route::get('/users', [AdminController::class, 'users']);
    Route::get('/posts', [AdminController::class, 'posts']);
});
// Genera: /admin/users y /admin/posts
```

Por middleware

```
Route::middleware(['auth'])->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
    Route::get('/profile', [ProfileController::class, 'show']);
});
```

Por nombre

```
Route::name('admin.')->group(function () {
    Route::get('/users', [AdminController::class, 'users'])->name('users');
    // Nombre completo: admin.users
});
```

3. Controladores

3.1 ¿Qué son los controladores?

Los controladores son clases que contienen la lógica de negocio de la aplicación. Son el punto de entrada desde el enrutador y actúan como intermediarios entre las peticiones del usuario, los modelos (base de datos) y las vistas (presentación).

En la arquitectura MVC (Modelo-Vista-Controlador), los controladores:

- Reciben las peticiones del usuario a través del enrutador
- Procesan la lógica necesaria (consultando modelos si es necesario)
- Devuelven una respuesta (generalmente una vista o datos JSON)

3.2 Principios de diseño de controladores

Un buen controlador debe ser:

1. **Delgado:** La lógica compleja debe estar en los modelos, no en los controladores.
2. **Organizado:** Cada método debe tener una responsabilidad clara.
3. **Sin acceso directo a la base de datos:** Usa modelos Eloquent para esto.
4. **Sin generación de HTML:** Usa vistas Blade para la presentación.

El controlador es un organizador del flujo, no un ejecutor de toda la lógica.

3.3 Estructura básica de un controlador

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index()
    {
        // Lógica para mostrar lista de usuarios
        return view('users.index');
    }
}
```



```

    }

    public function show($id)
    {
        // Lógica para mostrar un usuario específico
        return view('users.show', ['id' => $id]);
    }
}

```

3.4 Características importantes de los controladores

1. **Herencia:** Todos los controladores heredan de `Controller` o una subclase de `Controller`.
2. **Nomenclatura:**
 - o Singular
 - o CamelCase
 - o Termina en "Controller"
 - o Ejemplos: `UserController`, `ProductController`, `OrderController`
3. **Return obligatorio:** Cada método debe terminar con un `return`. Lo que devuelva se convierte automáticamente en una respuesta HTTP 200, excepto arrays que se convierten en JSON.

3.5 Creación de controladores con Artisan

Controlador básico

```
php artisan make:controller UserController
```

Genera un controlador vacío con la estructura básica:

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    //
}

```

Controlador Resource (RESTful)

```
php artisan make:controller UserController --resource
```

Genera un controlador con los siete métodos estándar REST:

```
<?php
```

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index()
    {
        // Mostrar lista de todos los usuarios
    }

    public function create()
    {
        // Mostrar formulario de creación
    }

    public function store(Request $request)
    {
        // Guardar nuevo usuario
    }

    public function show($id)
    {
        // Mostrar un usuario específico
    }

    public function edit($id)
    {
        // Mostrar formulario de edición
    }

    public function update(Request $request, $id)
    {
        // Actualizar usuario
    }

    public function destroy($id)
    {
        // Eliminar usuario
    }
}
```

Controlador API

```
php artisan make:controller UserController --api
```

Similar al resource, pero sin los métodos `create()` y `edit()` porque una API no necesita mostrar formularios HTML:

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index()
    {
        // Devolver JSON con lista de usuarios
    }

    public function store(Request $request)
    {
        // Crear usuario desde JSON
    }

    public function show($id)
    {
        // Devolver JSON con usuario específico
    }

    public function update(Request $request, $id)
    {
        // Actualizar usuario desde JSON
    }

    public function destroy($id)
    {
        // Eliminar usuario
    }
}

```

3.6 Paso de datos desde controlador a vista

Hay varias formas de pasar datos a las vistas:

Método 1: Array asociativo

```

public function show($nombre)
{
    $data = [
        'nombre' => $nombre,
        'edad' => 25,
        'ciudad' => 'Madrid'
    ];
    return view('usuario.perfil', $data);
}

```

En la vista:

```
<p>Nombre: {{ $nombre }}</p>
<p>Edad: {{ $edad }}</p>
<p>Ciudad: {{ $ciudad }}</p>
```

Método 2: Método with()

```
public function show($nombre)
{
    return view('usuario.perfil')
        ->with('nombre', $nombre)
        ->with('edad', 25);
}
```

Método 3: Método compact()

```
public function show($nombre)
{
    $edad = 25;
    $ciudad = 'Madrid';
    return view('usuario.perfil', compact('nombre', 'edad', 'ciudad'));
}
```

3.7 Tipos de respuestas desde controladores

Respuesta de vista

```
return view('welcome');
```

Respuesta de texto plano

```
return "Hola, mundo";
```

Respuesta JSON (automática para arrays)

```
return ['nombre' => 'Juan', 'edad' => 30];
// Laravel lo convierte automáticamente a JSON
```

Respuesta JSON explícita

```
return response()->json(['success' => true, 'data' => $datos]);
```

Redirección

```
return redirect('/home');
return redirect()->route('home');
return redirect()->back(); // Volver a la página anterior
```

Respuesta con código de estado personalizado

```
return response('No encontrado', 404);
return response()->json(['error' => 'No encontrado'], 404);
```

4. Arquitectura REST y Servidores RESTful

4.1 ¿Qué es REST?

REST (Representational State Transfer) es un estilo de arquitectura para diseñar servicios web. Un servidor RESTful sigue las convenciones REST para realizar operaciones CRUD (Create, Read, Update, Delete) sobre recursos.

Recursos: Cualquier cosa que se almacene en el servidor (usuarios, productos, pedidos, etc.). Generalmente corresponden a tablas de la base de datos.

4.2 Las 7 operaciones REST estándar

Para cada recurso, REST define 7 operaciones estándar:

Método	Ruta	Acción	Propósito
GET	/user	index	Listar todos los usuarios
GET	/user/{id}	show	Mostrar un usuario específico
GET	/user/create	create	Mostrar formulario de creación
POST	/user	store	Guardar nuevo usuario
GET	/user/{id}/edit	edit	Mostrar formulario de edición
PUT/PATCH	/user/{id}	update	Actualizar usuario
DELETE	/user/{id}	destroy	Eliminar usuario

4.3 Definición manual de rutas REST

```
Route::get('user', [UserController::class, 'index'])->name('user.index');
Route::get('user/create', [UserController::class, 'create'])->name('user.create');
Route::post('user', [UserController::class, 'store'])->name('user.store');
Route::get('user/{user}', [UserController::class, 'show'])->name('user.show');
Route::get('user/{user}/edit', [UserController::class, 'edit'])->name('user.edit');
Route::patch('user/{user}', [UserController::class, 'update'])->name('user.update');
Route::delete('user/{user}', [UserController::class, 'destroy'])->
    name('user.destroy');
```

4.4 Ruta resource (forma abreviada)

Laravel permite definir las 7 rutas REST con una sola línea:

```
Route::resource('user', UserController::class);
```

Esto es equivalente a las 7 rutas anteriores. Laravel automáticamente:

- Crea las 7 rutas con los nombres correctos
- Asocia cada ruta con el método correcto del controlador
- Asigna nombres estándar a cada ruta (`user.index` , `user.create` , etc.)

4.5 Ruta apiResource

Para APIs que no necesitan formularios HTML:

```
Route::apiResource('user', UserController::class);
```

Genera solo 5 rutas (excluye `create` y `edit`).

4.6 Restricción de rutas resource

Si no necesitas todas las rutas, puedes especificar cuáles incluir o excluir:

```
// Solo estas rutas
Route::resource('user', UserController::class)->only(['index', 'show']);

// Todas excepto estas
Route::resource('user', UserController::class)->except(['destroy']);
```

4.7 Ventajas de seguir REST

1. **Estandarización:** Cualquier desarrollador familiarizado con REST sabrá cómo usar tu API.
2. **Predictibilidad:** Las URLs y métodos son consistentes.
3. **Documentación implícita:** No necesitas explicar cada endpoint.
4. **Escalabilidad:** Fácil de extender con nuevos recursos.
5. **Interoperabilidad:** Otras aplicaciones pueden consumir tu API fácilmente.

5. Vistas con Blade

5.1 Ubicación de las vistas

Las vistas se almacenan en el directorio `/resources/views/` y deben tener la extensión `.blade.php` .

5.2 Retornar una vista desde el controlador

```
public function show($nombre)
{
    $data = ['nombre' => $nombre];
```

```
return view('usuarios.perfil', $data);  
}
```

Laravel buscará el archivo `/resources/views/usuarios/perfil.blade.php`.

5.3 Sintaxis básica de Blade

Mostrar variables

```
<p>Hola, {{ $nombre }}</p>
```

Las dobles llaves `{{ }}` escapan automáticamente el contenido HTML para prevenir ataques XSS.

Mostrar HTML sin escapar (usar con precaución)

```
<div>{!! $contenidoHTML !!}</div>
```

Estructuras de control

```
@if($edad >= 18)  
    <p>Eres mayor de edad</p>  
@else  
    <p>Eres menor de edad</p>  
@endif  
  
@foreach($usuarios as $usuario)  
    <li>{{ $usuario->nombre }}</li>  
@endforeach  
  
@for($i = 0; $i < 10; $i++)  
    <p>Número: {{ $i }}</p>  
@endfor
```

6. Configuración inicial de una aplicación Laravel

6.1 Generar clave de aplicación

```
php artisan key:generate
```

Este es el primer comando que debes ejecutar al configurar una nueva aplicación. Genera una clave de 32 caracteres que se usa para:

- Cifrar sesiones
- Hashear contraseñas
- Generar tokens CSRF
- Cualquier otra operación de encriptación

Sin esta clave, Laravel lanzará errores y no funcionará correctamente.

6.2 Configurar el archivo .env

El archivo `.env` contiene las variables de configuración específicas del entorno:

```
APP_NAME=MiaAplicacion
APP_ENV=local
APP_KEY=base64:...
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=nombre_base_datos
DB_USERNAME=usuario
DB_PASSWORD=contraseña
```

6.3 Configurar la base de datos

Después de configurar `.env`, ejecuta:

```
php artisan migrate
```

Esto creará todas las tablas necesarias en la base de datos según las migraciones definidas.

7. Mejores prácticas y consejos

7.1 Nomenclatura consistente

- **Controladores:** Singular, CamelCase, terminan en Controller
 - `UserController`, `ProductController`
- **Modelos:** Singular, CamelCase
 - `User`, `Product`
- **Tablas de base de datos:** Plural, snake_case
 - `users`, `products`, `order_items`
- **Rutas:** Plural, kebab-case
 - `/users`, `/products`, `/order-items`

7.2 Mantén los controladores delgados

Mueve la lógica compleja a:

- **Modelos:** Lógica relacionada con datos
- **Servicios:** Lógica de negocio compleja
- **Jobs:** Tareas que pueden ejecutarse en segundo plano
- **Events y Listeners:** Lógica activada por eventos

7.3 Usa validación de formularios

No valides datos directamente en el controlador. Crea Form Requests:


```
php artisan make:request StoreUserRequest
```

7.4 Protección CSRF

Todos los formularios POST, PUT, PATCH y DELETE deben incluir el token CSRF:

```
<form method="POST" action="/usuario">
  @csrf
  <!-- campos del formulario -->
</form>
```

7.5 Consulta la documentación

Laravel tiene una documentación excelente en <https://laravel.com/docs>. Consúltala regularmente para aprovechar todas las características del framework.

8. Resumen de flujo de trabajo típico

1. **Crear una migración** para la tabla:

```
php artisan make:migration create_users_table
```

2. **Ejecutar la migración:**

```
php artisan migrate
```

3. **Crear un modelo:**

```
php artisan make:model User
```

4. **Crear un controlador resource:**

```
php artisan make:controller UserController --resource
```

5. **Definir las rutas** en `web.php` :

```
Route::resource('usuarios', UserController::class);
```

6. **Implementar los métodos** del controlador

7. **Crear las vistas** necesarias en `/resources/views`

8. **Probar la aplicación** usando `php artisan serve`